IoT Systems Lab 1:

IoT Messaging Platform Activity

Overview

In **Internet of Things (IoT)** systems, devices often communicate using the **Publish/Subscribe (Pub/Sub) messaging model** such as Message Queuing Telemetry Transport (MQTT) which is one of the most widely used messaging protocols. The Pub/Sub model consists of three main components:

1. **Publisher:** Creates and publishes messages to a topic.
2. **Broker:** Receives messages from publishers and forwards them to subscribers based on their topic subscriptions.
3. **Subscriber:** Subscribes to specific topics to receive and processes incoming messages.

This architecture **decouples the data senders and receivers** from being directly connected. This allows devices to exchange information reliably, **asynchronously**, and at scale. Since publishers and subscribers do not depend on each other, pub/sub systems are highly flexible, scalable, and widely used in IoT platforms.

Objective

The goal of this activity is to gain **practical experience with MQTT** by implementing basic communication using the Publish/Subscribe (Pub/Sub) model. You will develop **a publisher and a subscriber** that exchange messages through a message broker.

Part 1: Basic Publisher and Subscriber

In this part, students have to implement a simple MQTT publisher and consumer to send device MAC addresses to the provided broker and receive responses in return.

Your Work

1. Implement a publisher and a subscriber in *any* programming language of your choice.
2. Access to the provided broker (Link will be announced during class).
3. Publish messages (payload) to topic "**IotMsgPlatformPartOne**" with subtopic of **your group name**.
   3.1. Subtopic
   Use your group name as a subtopic. Go to Student Group on mCV to find your group name, displayed in the format "**Gxx: {YourGroupName}**". Use **{YourGroupName}** as a subtopic.
   - Note: If your group name contains the special characters `#` or `+`, remove them for MQTT subtopic. For example your group name on mCV is "G67: MQTT #1" the subtopic is "MQTT 1".

   3.2. Payload
   In the payload, you must submit **your MAC address** of a network interface from your computer used for the activity, e.g., WiFi, en0.
   - How to locate your MAC address
     - https://oit.ua.edu/services/internet-networking/finding-your-mac-address/
     - https://www.dell.com/support/kbdoc/en-th/000123355/how-to-determine-the-mac-address-of-a-network-adapter-in-windows

   3.3. Information format to publish your MQTT message.

Topic: **`IotMsgPlatformPartOne/{YourGroupName}`**
`{ "mac_address": "AA:BB:CC:DD:EE:FF" }`

4. After messages published to the topic **"IotMsgPlatformPartOne/*"**, you will receive responses on the topic **"IotMsgPlatformPartOne/respond"**, containing the following payload.

Topic: **`IotMsgPlatformPartOne/respond`**

`{`
`  "group_id": "IoT",`
`  "v2x_topic": "IotTopicPartTwo",`
`  "error": ""`
`}`

- **`group_id`**: The ID of the group to which this message belongs.
- **`v2x_topic`**: This topic will be used in **Part 2**, which is **unique for each group**.
- **`error`**: The explanations show whether an error occurred.

IoT Systems Lab1: IoT Messaging Platform (Part 1) Assignment

myCourseVille Questions

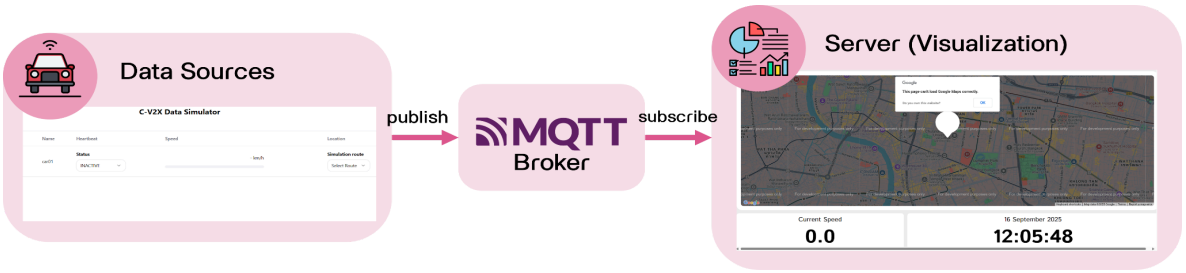
1. What is the group number and group name (as listed in Student Group on mCV) ?
2. What is the MAC address of the network interface of the computer used for the activity ?
3. What is the v2x_topic returned by the broker ?

myCourseVille Attachment

1. MAC Address Screenshot (Entire screen must be visible)

Part 2: IoT Messaging Platform in Internet of Vehicles (IoV)

In this part, students will explore how an IoT messaging platform can be applied in Internet of Vehicles (IoV) scenarios, where a connected vehicle transmitted sensor data over a network to be displayed on a monitoring platform, such as a dashboard. Some starter code will be provided; your task is to modify it so that the sensor data is sent through an MQTT broker and displayed on a web dashboard.



Prerequisites

1. Git and Node.js installed on your computer
2. **Deploy an MQTT broker** by hosting your own broker locally or on a cloud server (e.g., Mosquitto, HiveMQ).
3. Clone the repository (git-url will be announced during class).

```
$ git clone <git-url>
```

4. Observe the repository. There are three main folders: **`Frontend`**, **`OBU`**, and **`Sensor`**.
5. Install dependencies for (1) **`Frontend`** (2) **`OBU`** (3) **`Sensor/server`** (4) **`Sensor/web`** using following command:

```
$ npm install
```

6. Configure environment variables
   - Copy environment template using following command:

```
$ cd Frontend && cp .env.local.example .env.local
```

```
$ cd OBU && cp .env.example .env
```

   - Modify the environment variables in **`Frontend/.env.local`** and **`OBU/.env`** and update the **`id`** variable in **`Sensor/server/repository/obu.js`** to match your environment configuration.

7. Run the activity

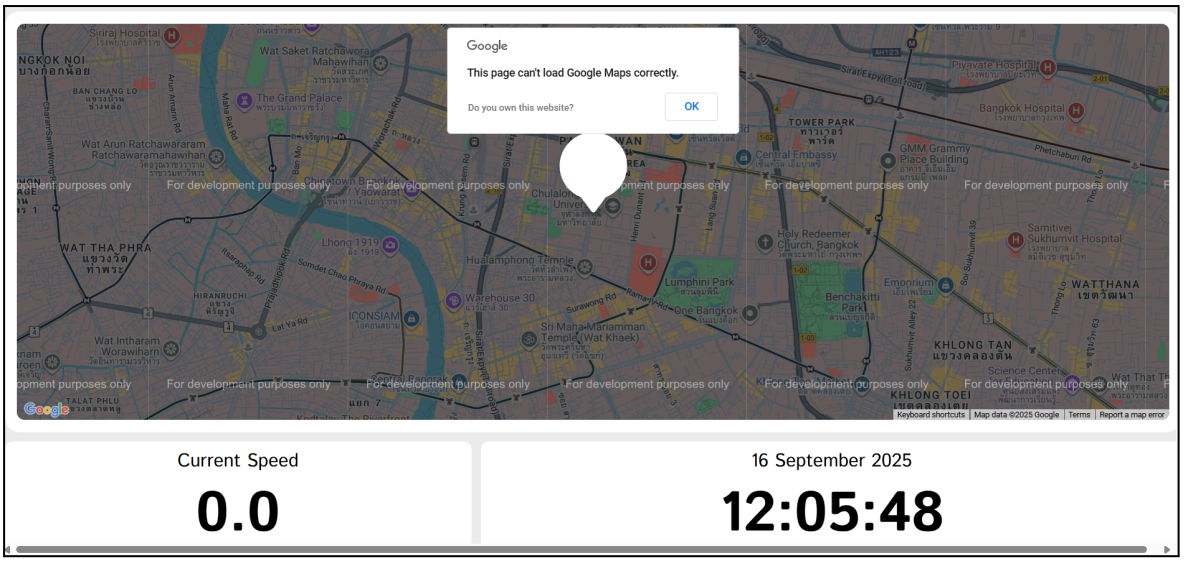
```
$ cd Frontend && npm run dev
```

```
$ cd OBU && npm run dev
```

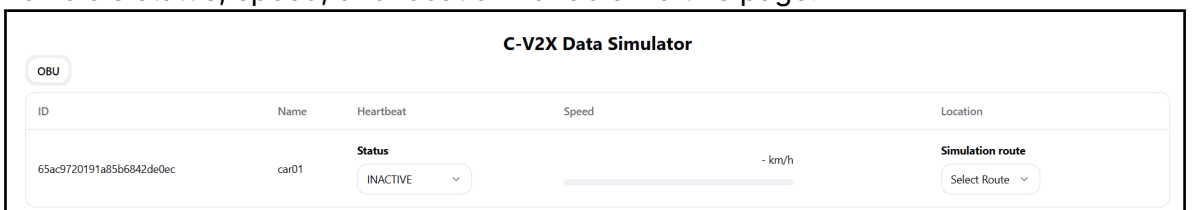
```
$ cd Sensor/server && npm run start:dev
```

If everything goes well, you should see the following pages:

- http://localhost:3000/dashboard: This page is used for *displaying* the vehicle's speed and location in real time. (Ignore "For development proposes only" watermark.)



- http://localhost:5173: This page is used as a *simulator*. You can adjust the vehicle's status, speed, and location variable via this page.



  - **ID, Name**: Display id and name of the vehicle.
  - **Heartbeat**: Control active status of the vehicle.
    - **INACTIVE**: The vehicle goes inactive. When the vehicle is in **INACTIVE** status, the car's pin will not appear on the map, and its information will **NOT** be updated.
    - **ACTIVE**: The vehicle is active.
  - **Speed**: Control the vehicle's speed.
  - **Location**: Control the vehicle's location.
    - จุฬา: Simulate the vehicle's movement by publishing locations around Chulalongkorn University Square to the broker.
    - **Fixed**: Stop the vehicle's movement at a fixed location.

Your Work

Complete a publisher and subscriber using the provided code.

1. Publisher

Go to **`Sensor/server/obu.js`** and implement the publisher by filling in missing code in that file. This file is responsible for receiving data from the simulator's web page and publishing them to subscribers via a broker.

⚠ The message must be published in the following format

Topic: **`<v2x_topic>/speed`**

```
{
  "speed": <speed-value>
}
```

Topic: **`<v2x_topic>/heartbeat`**

```
{
  "heartbeat": <true or false>
}
```

Topic: **`<v2x_topic>/route`**

```
{
  "latitude":  <latitude-value>,
  "longitude": <longitude-value>,
  "color":     <color-value>
}
```

2. Subscriber

Go to **`OBU/consumer.js`** and implement the subscriber to receive data from the broker. **Several necessary functions are provided at the top of the file**. Use these functions to process the subscribed data and display the simulated behavior of the vehicle on the dashboard.

ⓘ You may need to deploy a broker yourself to *test* the implemented publisher and subscriber.

After completing this part,
please contact TAs to review and grade your work.

IoT Systems Lab1: IoT Messaging Platform (Part 2) Assignment

myCourseVille Questions

1. What is the vehicle's speed?
2. What is the color of the car pin when it's moving from the Faculty of Science, Chulalongkorn University, to the Faculty of Architecture, Chulalongkorn University?
3. What is the color of the car pin when it's moving from the Faculty of Architecture, Chulalongkorn University, to the Faculty of Fine and Applied Arts, Chulalongkorn University?
4. What is the color of the car pin when it's moving from the Faculty of Art, Chulalongkorn University, to the Faculty of Engineering, Chulalongkorn University?
5. What is the color of the car pin when it's moving from the Faculty of Engineering, Chulalongkorn University, to the Faculty of Science, Chulalongkorn University?

myCourseVille Attachment / Textbox

1. Your source code (e.g., Zip file, Github URL, etc.)